

CUDA C++ By Example

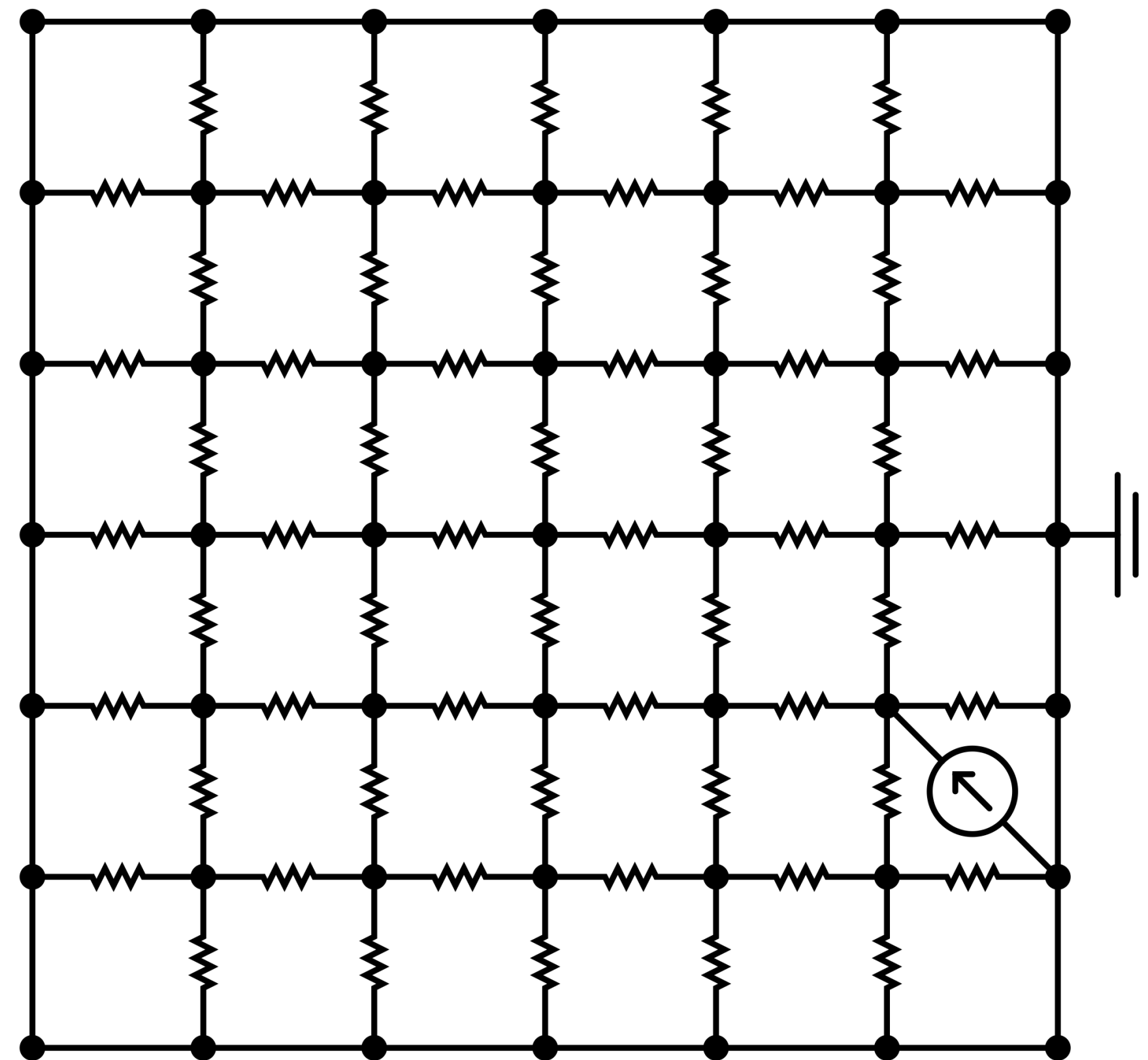
Review: Session 3

- Compilers
 - nvcc, clang, nvc++
- Build
 - CMake, ninja / ninjatrace, perfetto
- Debugging
 - compute-sanitizer, cuda-gdb
- Profiling Tools
 - NSight Systems
 - NSight Compute
 - NVTX
- Start on Kernel Optimization (time permitting)

Poisson's Equation / Resistor Network

$$\nabla^2 u = f, \quad u = 0 \text{ on } \partial\Gamma$$

Which, after discretization is isomorphic to



Poisson's Equation / Resistor Network

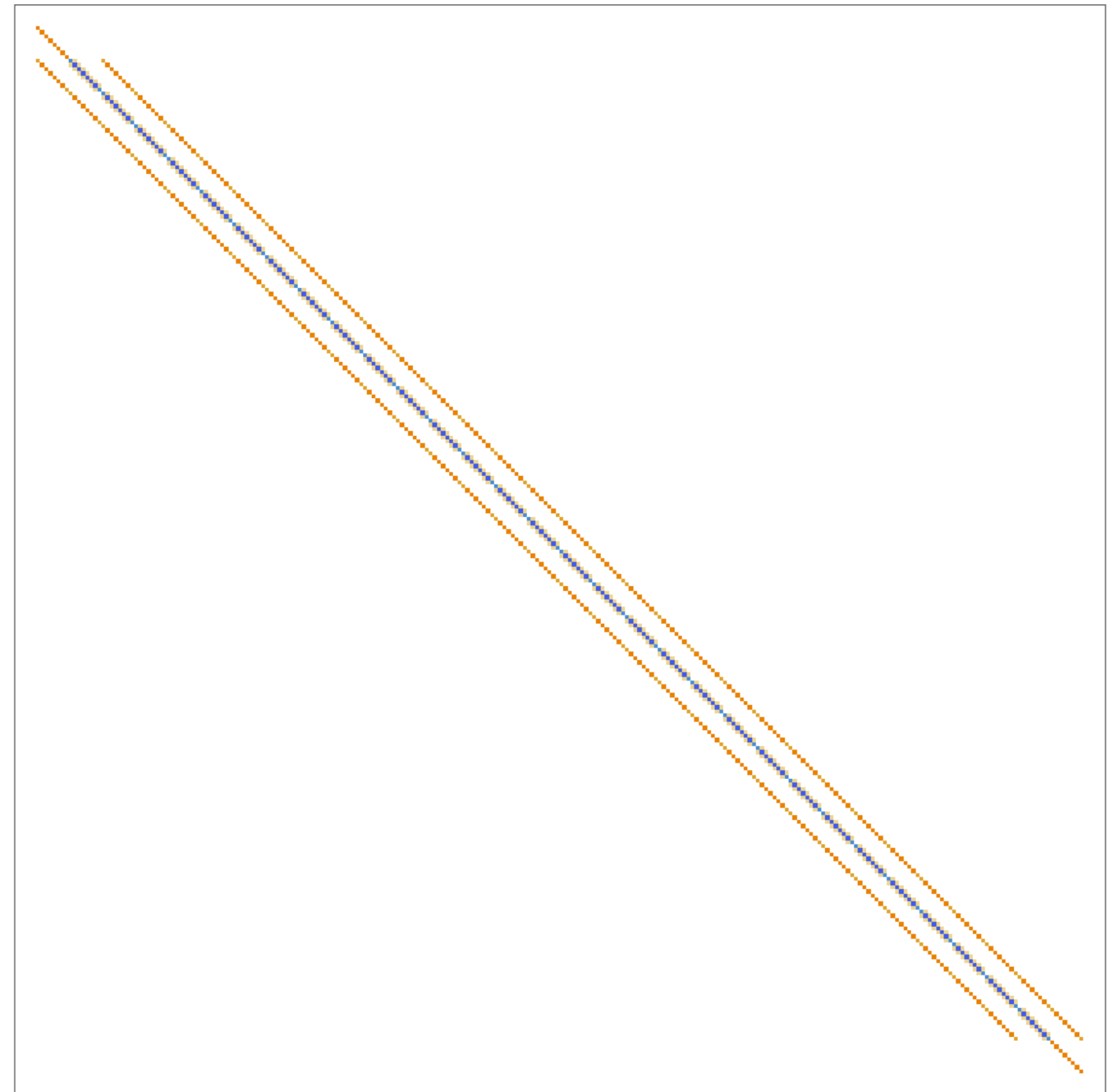
$$\nabla^2 u = f, \quad u = 0 \text{ on } \partial\Gamma$$

discretize the problem, express as a system of linear equations, and solve

$$\mathbf{A}\mathbf{x} = \mathbf{b}$$

or, equivalently

$$\mathbf{G}\mathbf{v} = \mathbf{i}$$

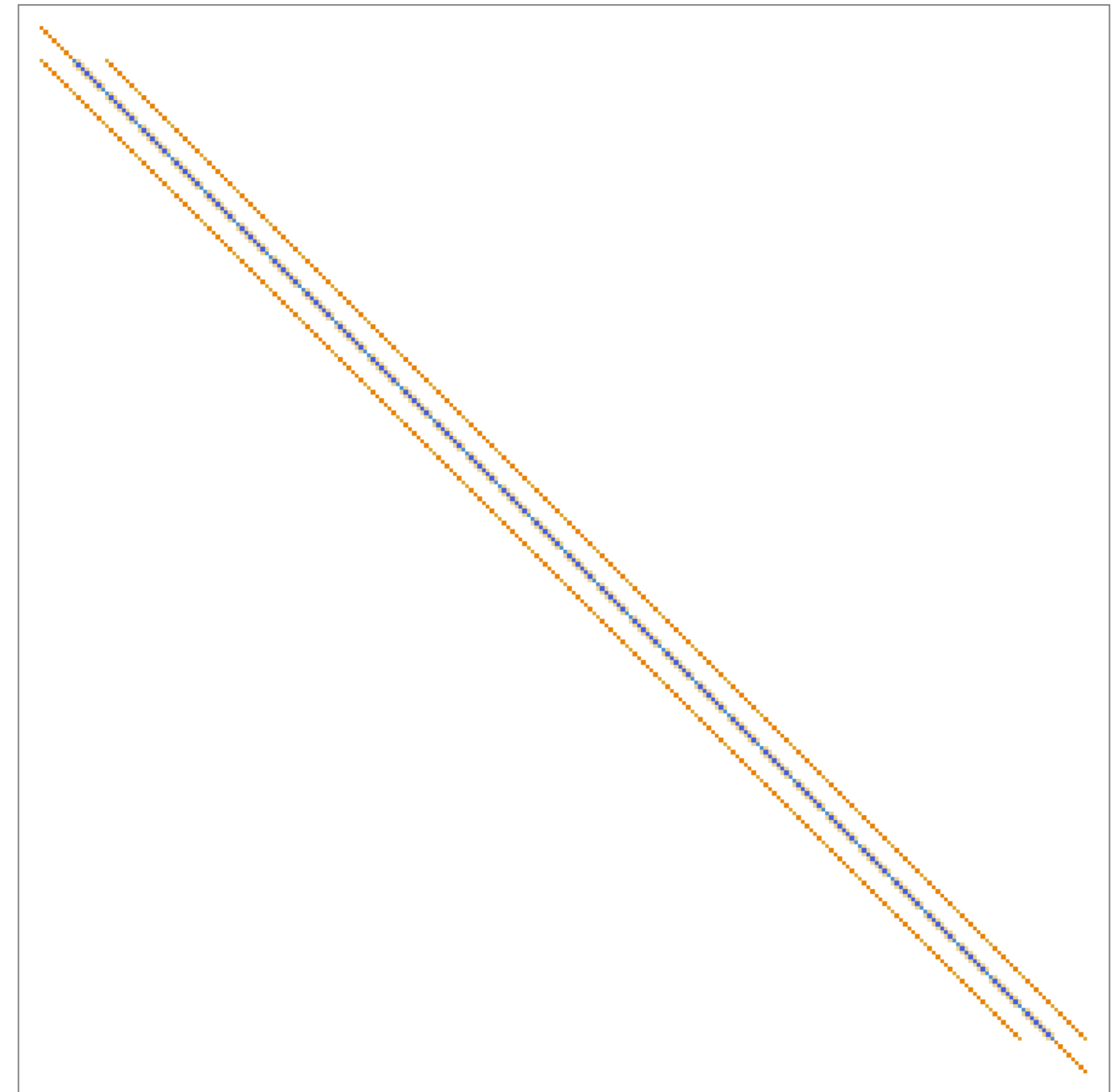


coefficient matrix is large, but **sparse**

Poisson's Equation / Resistor Network

Solution strategies:

- Direct Methods (e.g. $\mathbf{A} = \mathbf{LU}$)
 - Expensive upfront cost
 - Fill-in (factors may not be sparse)
 - Good for solving **many** r.h.s.
- Iterative Methods
 - Simple to implement
 - Small memory footprint
 - Good for solving **few** r.h.s.



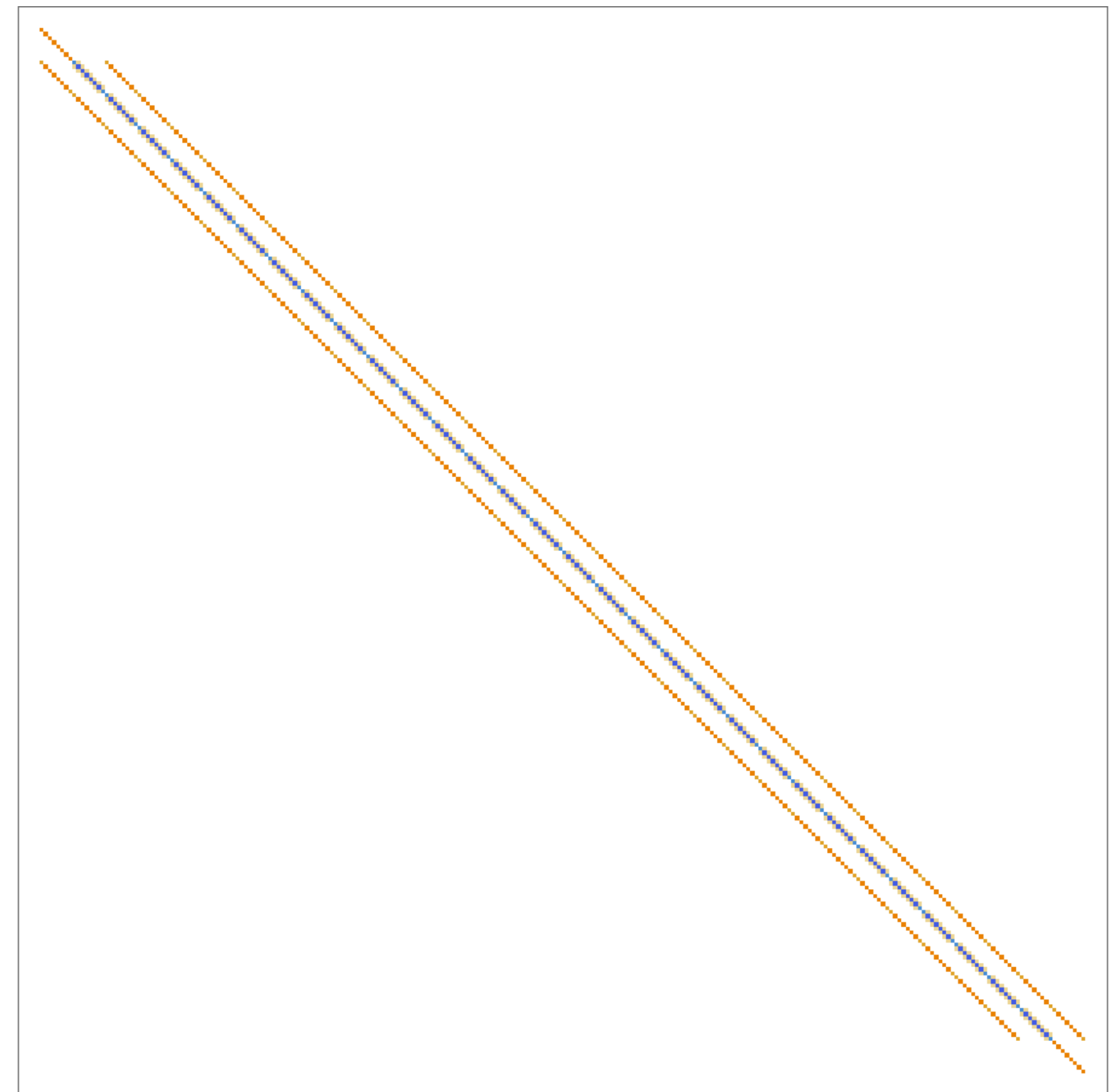
coefficient matrix is large, but **sparse**

Poisson's Equation / Resistor Network

Solution strategies:

- Direct Methods (e.g. $\mathbf{A} = \mathbf{LU}$)
 - Expensive upfront cost
 - Fill-in (factors may not be sparse)
 - Good for solving **many** r.h.s.
- Iterative Methods
 - Simple to implement
 - Small memory footprint
 - Good for solving **few** r.h.s.

Let's implement and optimize
an iterative solver in CUDA



coefficient matrix is large, but **sparse**

Jacobi / Relaxation Method

$$\nabla^2 u(x, y) \approx \frac{1}{\delta^2} \left(\begin{array}{c} +u(x, y+\delta) \\ +u(x-\delta, y) - 4u(x, y) + u(x+\delta, y) \\ +u(x, y-\delta) \end{array} \right) = f(x, y)$$



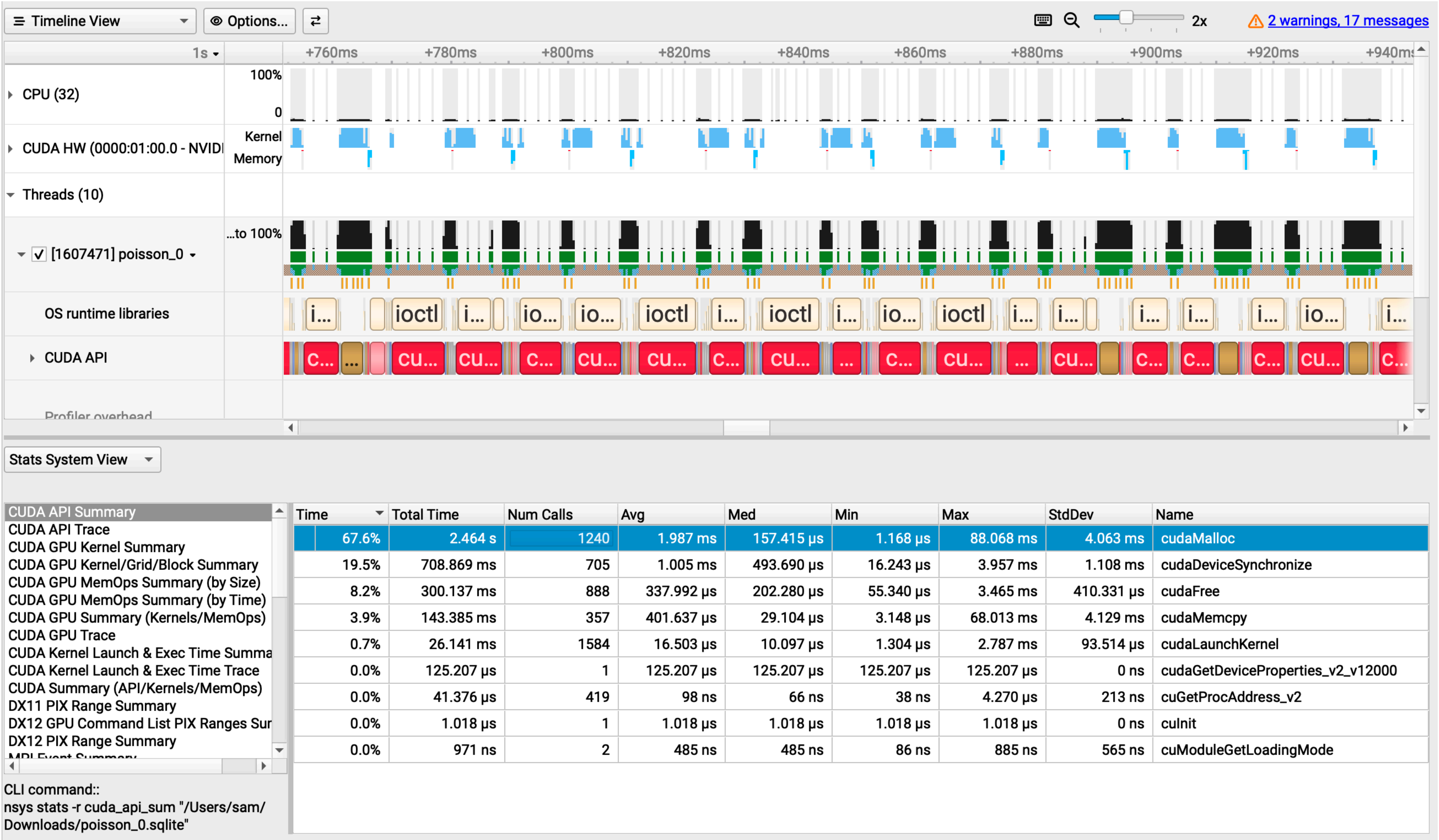
idea: isolate $u(x, y)$ term on one side,
and use this as a fixed-point iteration

$$u(x, y) = \frac{1}{4} \left(\begin{array}{c} +u(x, y+\delta) \\ +u(x-\delta, y) - \delta^2 f(x, y) + u(x+\delta, y) \\ +u(x, y-\delta) \end{array} \right)$$

Jacobi / Relaxation Method

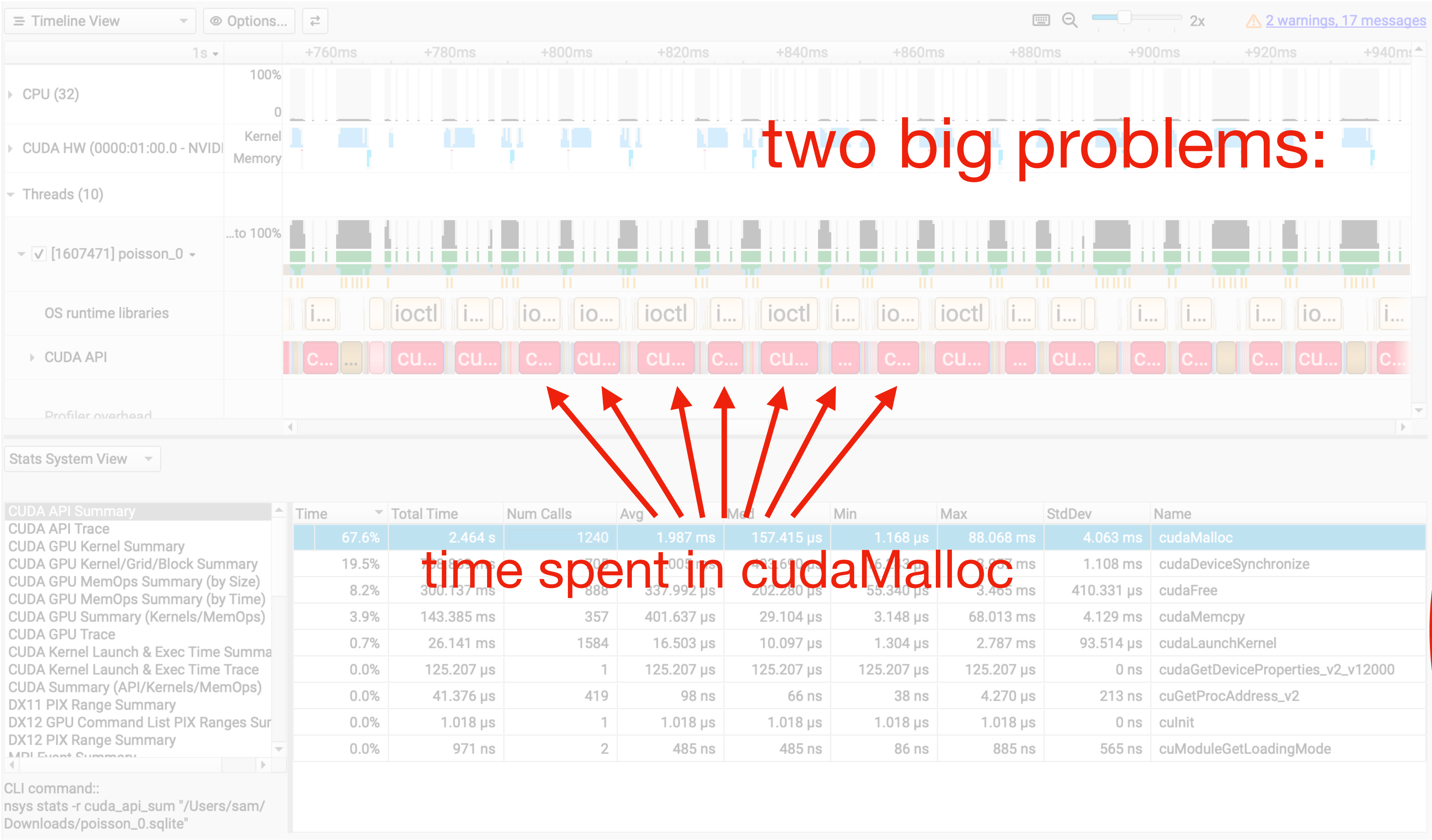
Brief Code Walkthrough
(see: [06_profiling/poisson_0](#))

Jacobi / Relaxation Method



```
~/code/Intro-to-CUDA/build$ ./poisson_0
ndof: 16777216, nnz: 98712536
0 0.408248
1 0.263523
2 0.199665
3 0.16319
4 0.139255
5 0.122182
6 0.109311
7 0.0992158
8 0.091058
9 0.0843111
10 0.0786259
...
163 0.0105274
164 0.0104795
165 0.0104322
166 0.0103854
167 0.0103391
168 0.0102932
169 0.0102479
170 0.0102029
171 0.0101585
172 0.0101145
173 0.0100709
174 0.0100278
175 0.00998508
finished in 3478.52 ms
```

Jacobi / Relaxation Method



two big problems:

time spent in cudaMalloc

```
~/code/Intro-to-CUDA/build$ ./poisson_0
ndof: 16777216, nnz: 98712536
```

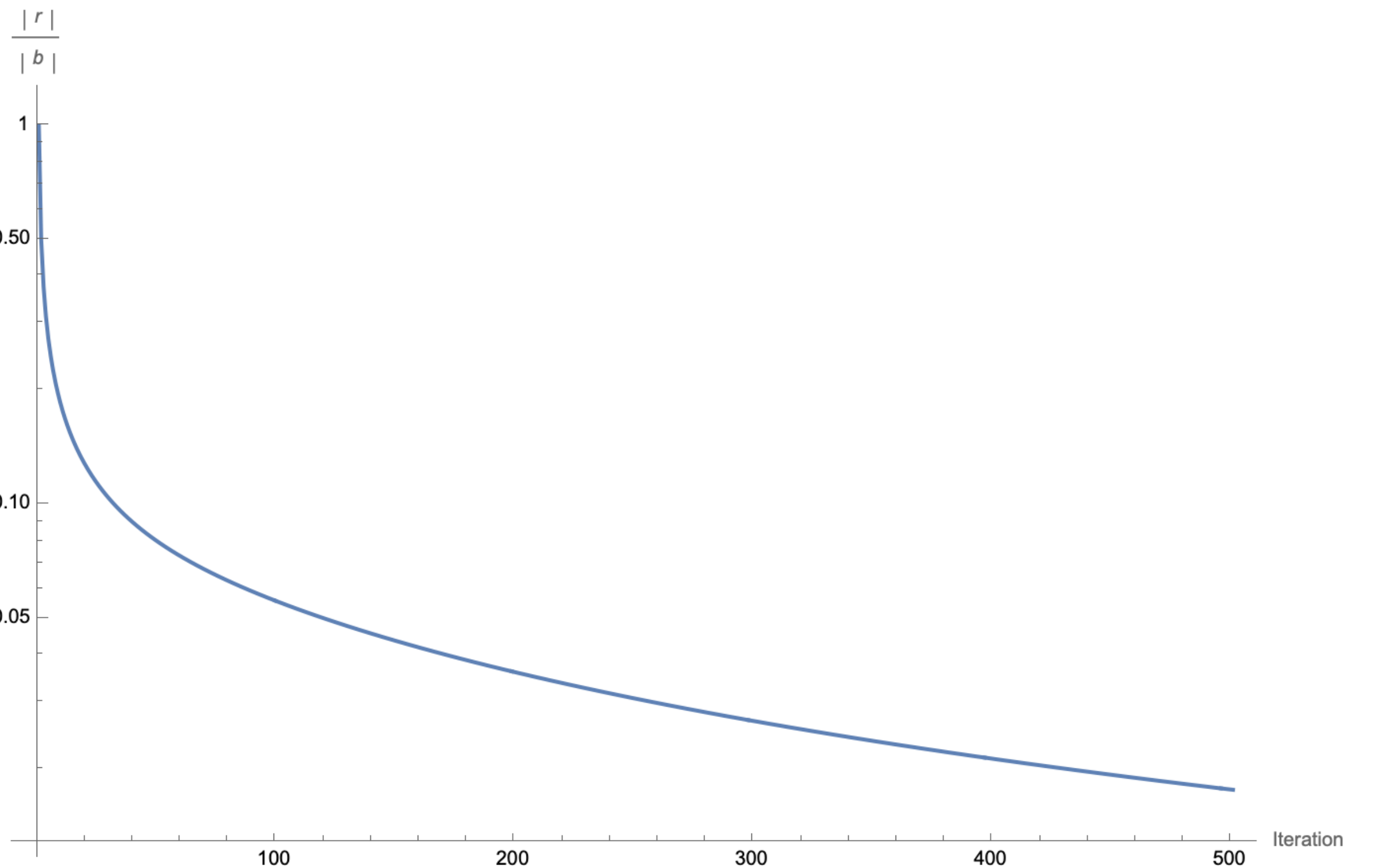
```
0 0.408248
1 0.263523
2 0.199665
3 0.16319
4 0.139255
5 0.122182
6 0.109311
7 0.0992158
8 0.091058
9 0.0843111
10 0.0786259
```

```
...
153 0.0105274
164 0.0104795
165 0.0104322
166 0.0103854
167 0.0103391
168 0.0102932
169 0.0102479
170 0.0102029
171 0.0101585
172 0.0101145
173 0.0100709
174 0.0100278
175 0.00998508
```

finished in 3478.52 ms

slow convergence

Optimization #1: Algorithm



Jacobi Method converges very slowly

Optimization #1: Algorithm

Let's try the **conjugate gradient** algorithm instead:

- Better convergence rate
- Requires symmetry, positive definiteness
- Simple to implement
 - provided in poisson_1/cg.hpp
- Can be improved further with preconditioning

```

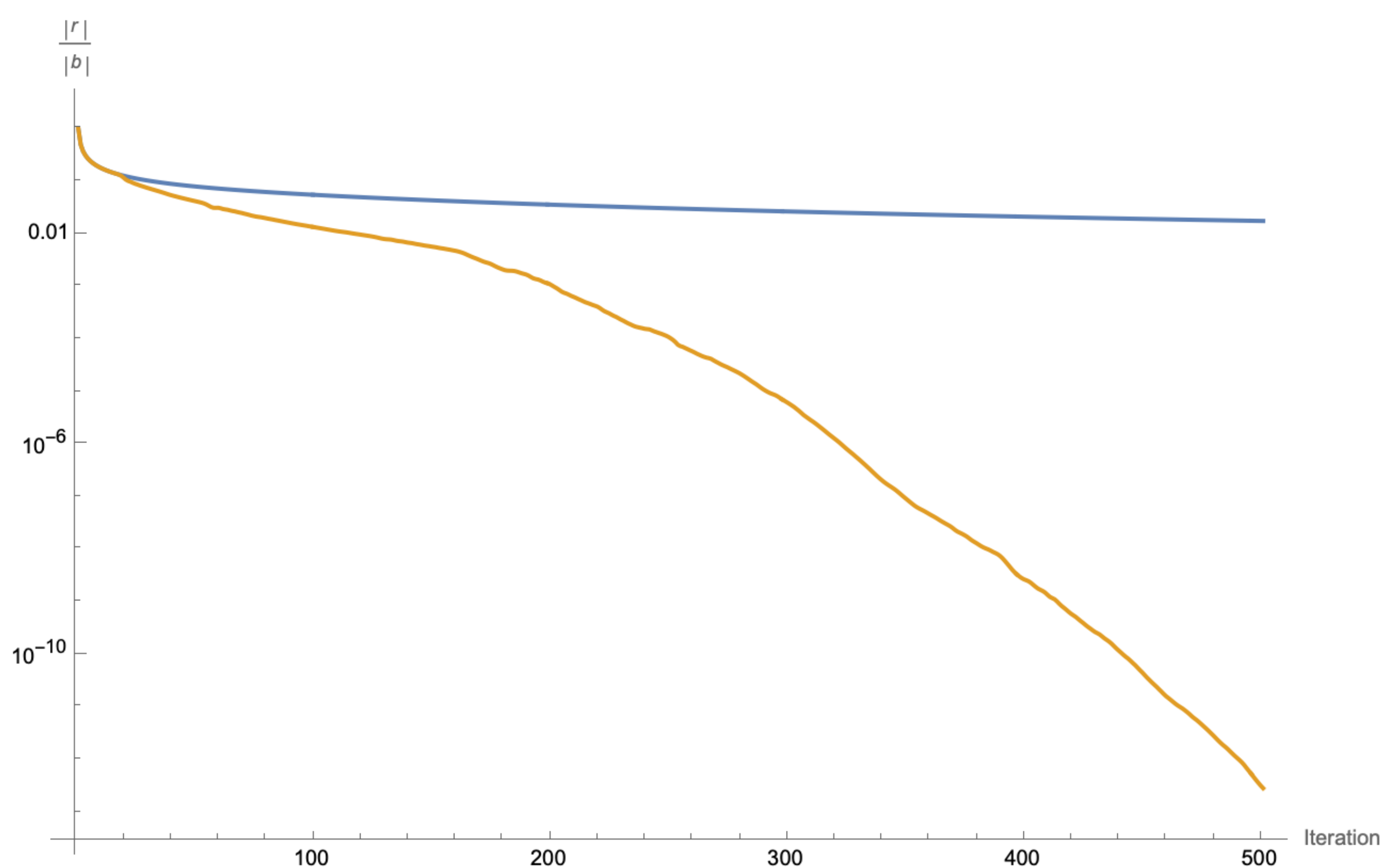
$$\begin{aligned} i &\leftarrow 0 \\ r &\leftarrow b - Ax \\ d &\leftarrow r \\ \delta_{new} &\leftarrow r^T r \\ \delta_0 &\leftarrow \delta_{new} \\ \text{While } i < i_{max} \text{ and } \delta_{new} > \varepsilon^2 \delta_0 \text{ do} \\ &\quad q \leftarrow Ad \\ &\quad \alpha \leftarrow \frac{\delta_{new}}{d^T q} \\ &\quad x \leftarrow x + \alpha d \\ &\quad \text{If } i \text{ is divisible by } 50 \\ &\quad \quad r \leftarrow b - Ax \\ &\quad \text{else} \\ &\quad \quad r \leftarrow r - \alpha q \\ &\quad \delta_{old} \leftarrow \delta_{new} \\ &\quad \delta_{new} \leftarrow r^T r \\ &\quad \beta \leftarrow \frac{\delta_{new}}{\delta_{old}} \\ &\quad d \leftarrow r + \beta d \\ &\quad i \leftarrow i + 1 \end{aligned}$$

```

Optimization #1: Algorithm

Brief Code Walkthrough
(see: 06_profiling/poisson_1)

Optimization #1: Algorithm



```
~/code/Intro-to-CUDA/build$ ./poisson_1
0 0.166667
1 0.06
2 0.032124
3 0.0195303
4 0.0131258
5 0.00953437
6 0.00713249
7 0.00559492
8 0.00448975
9 0.00367153
10 0.00307865
...
53 0.000138708
54 0.00013376
55 0.000129046
56 0.00012465
57 0.000120391
58 0.000116405
59 0.000112602
60 0.000108948
61 0.00010553
62 0.000102211
63 9.90787e-05
finished in 1516.11 ms
```

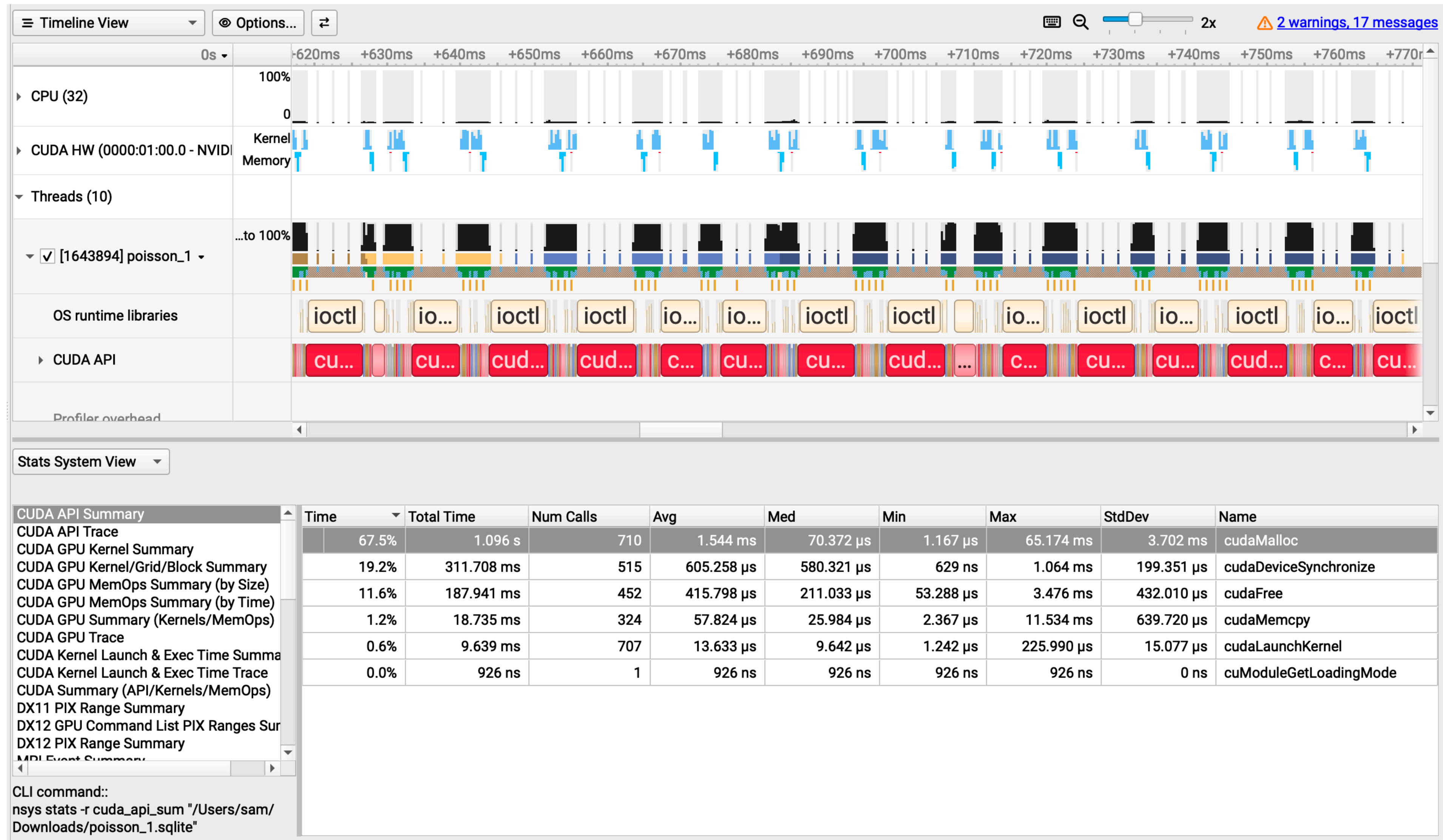
Optimization #1: Algorithm



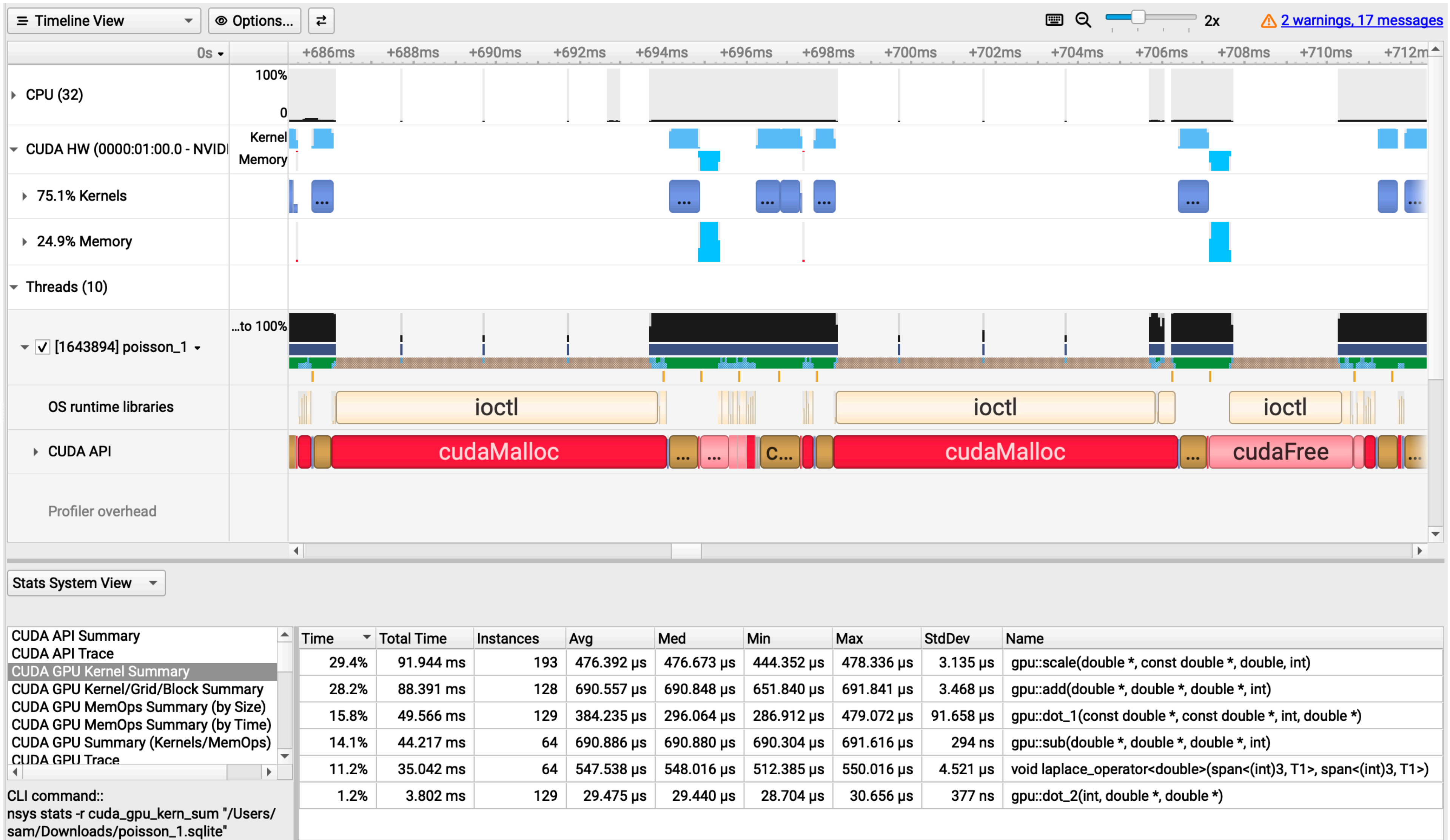
Jacobi Method

Conjugate Gradient

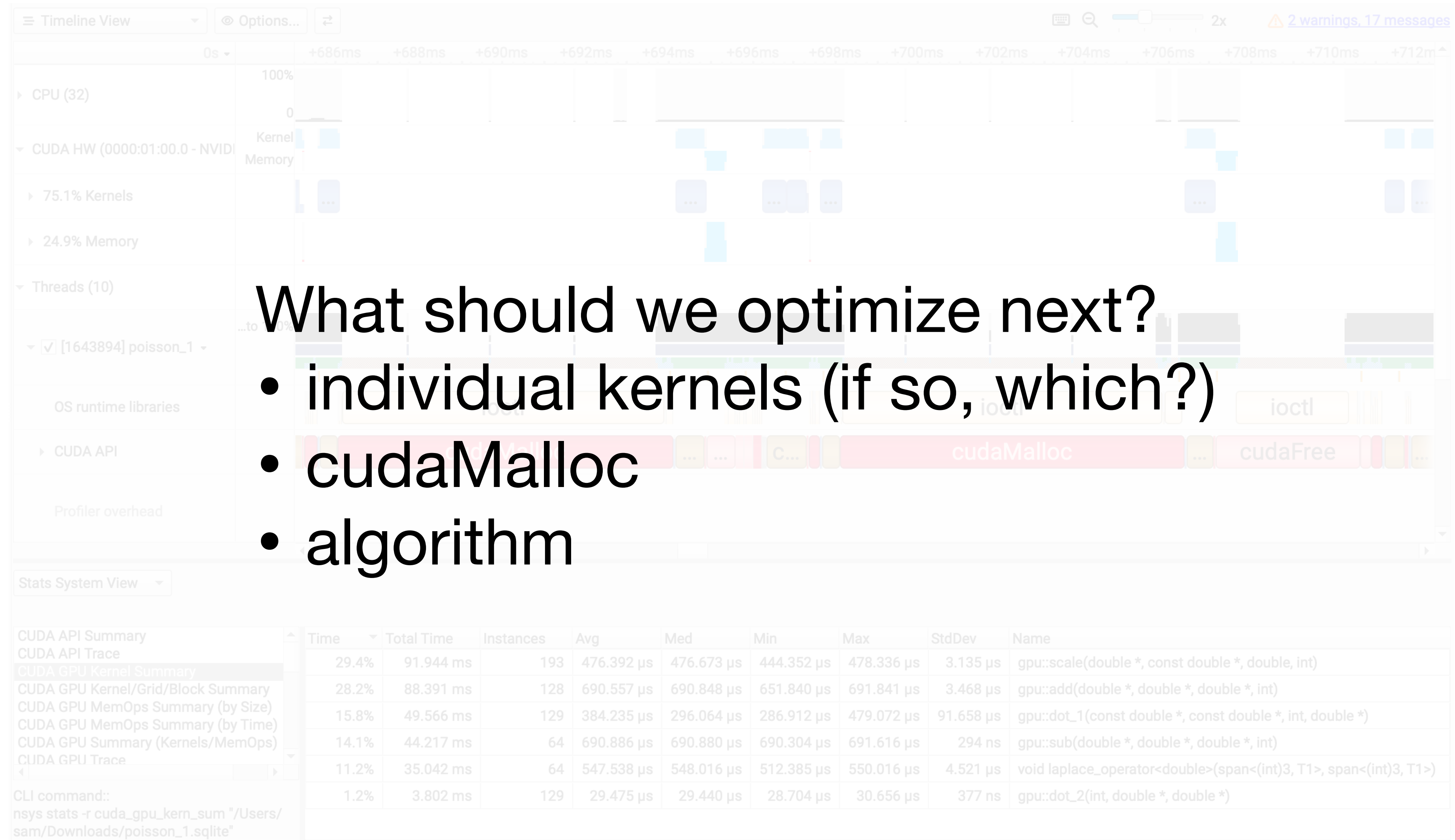
Optimization #1: Algorithm



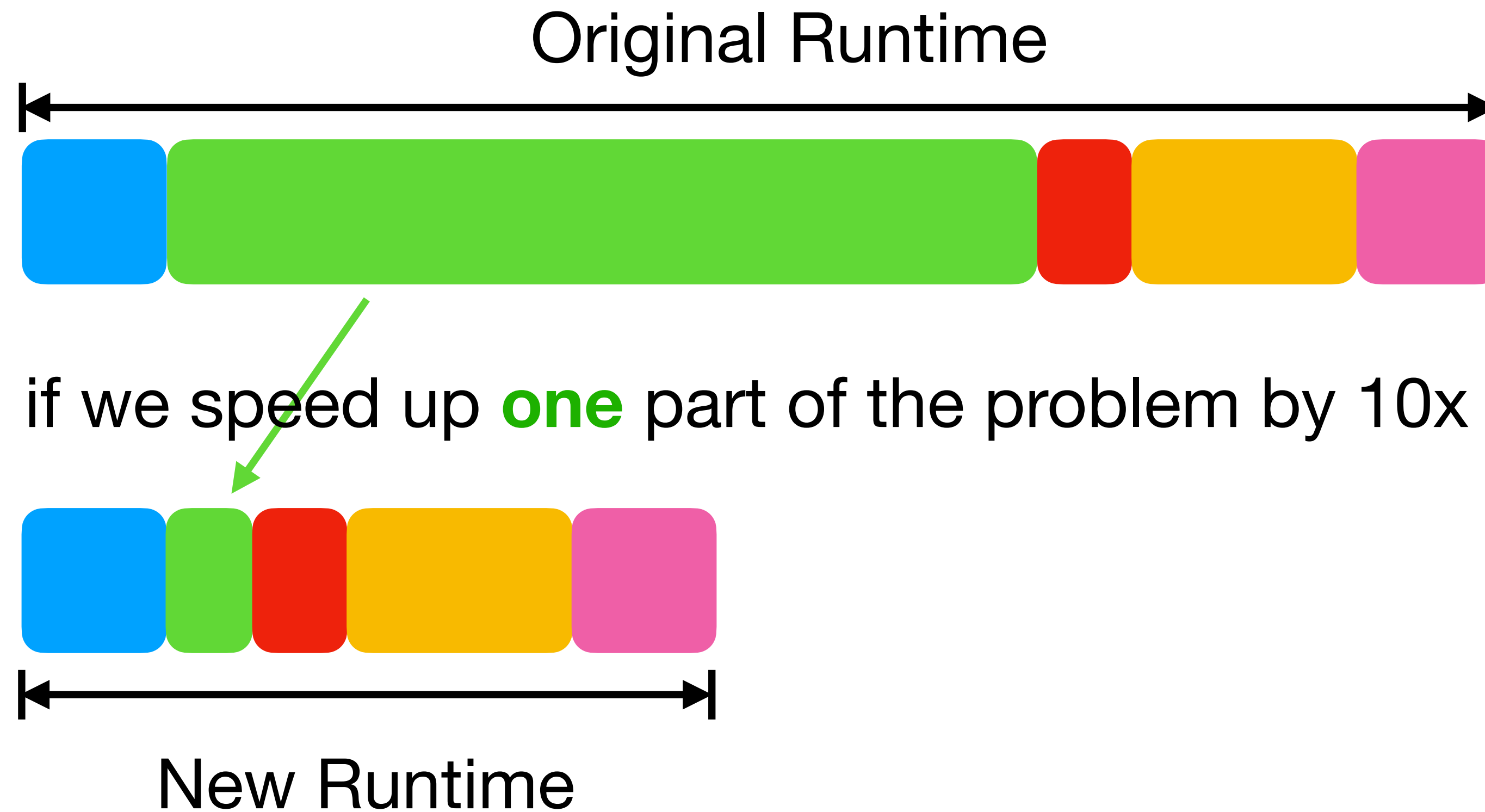
Optimization #1: Algorithm



Optimization #1: Algorithm



Aside: Amdahl's Law

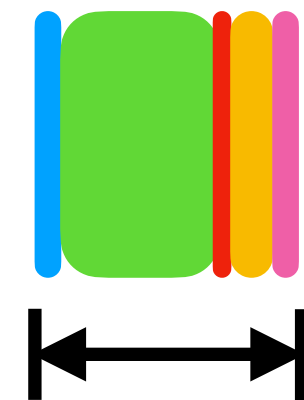


Overall Speedup is **less than** 10x

Aside: Amdahl's Law



if we speed up **every** part of the problem by 10x



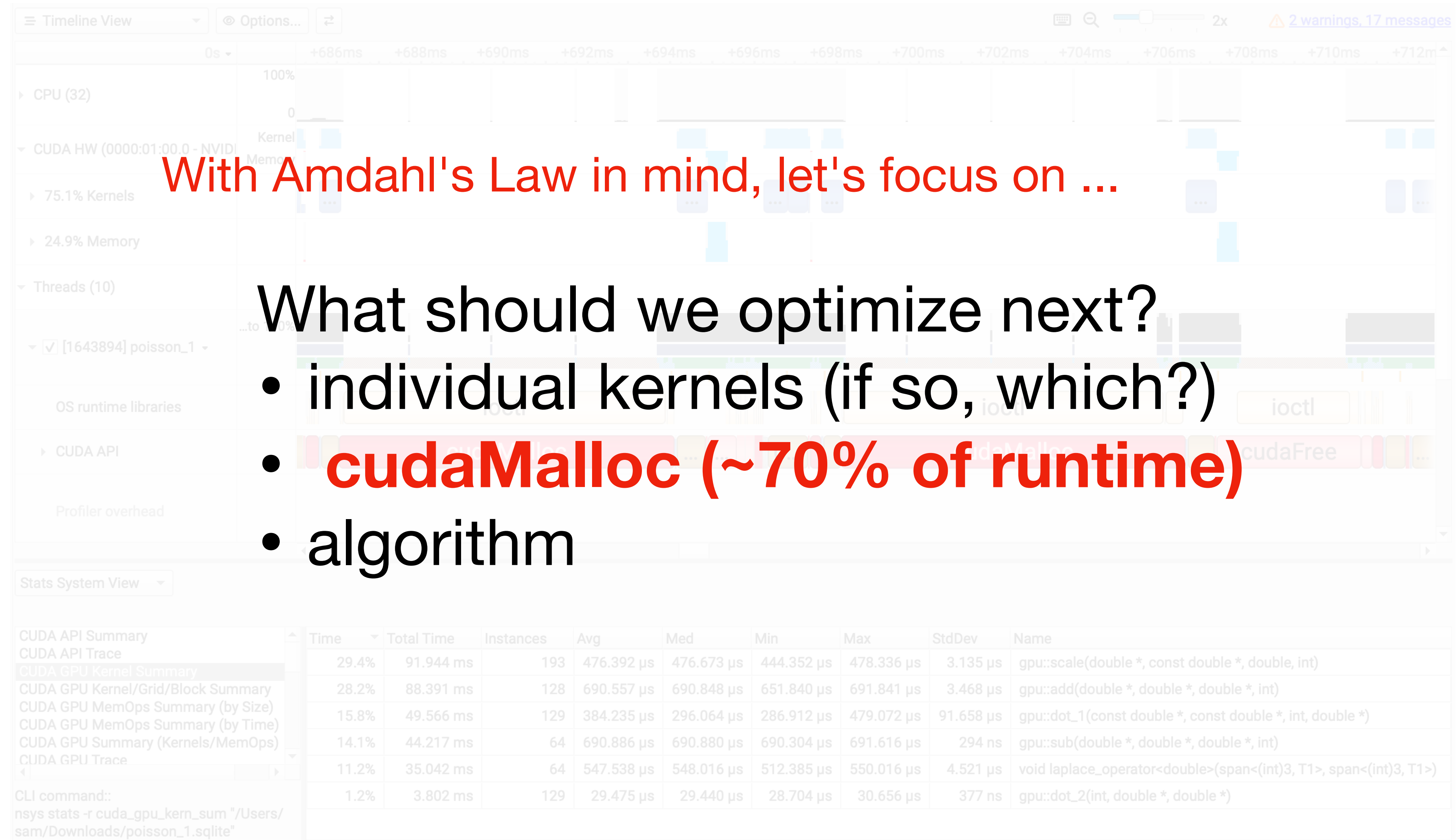
New Runtime

Overall Speedup is 10x

Aside: Amdahl's Law

- N-fold speedup of one part $\neq$ N-fold speedup overall
 - N-fold speedup of **every** part $==$ N-fold speedup overall
- Generally: try to **focus on the parts that take a long time**, since they have the biggest potential for overall speedup
 - Note: some kernels may already be reasonably optimized, but still take a long time.
- Corollary: "tiny" kernels that don't contribute significantly to overall runtime don't *need* to be optimal

Optimization #1: Algorithm



Optimization #2: Memory Allocation

```
int i = 0;
while (i < imax && delta > ((epsilon * epsilon) * delta0)) {
    VectorType q = A(d);
    double alpha = delta / dot(d, q);
    x = x + alpha * d;

    r = r - alpha * q;

    double delta_old = delta;
    delta = dot(r, r);

    double beta = delta / delta_old;
    d = r + beta * d;
    i++;
}
```

cg.hpp

Optimization #2: Memory Allocation

```
int i = 0;
while (i < imax && delta > ((epsilon * epsilon) * delta0)) {
    VectorType q = A(d);
    double alpha = delta / dot(d, q);
    x = x + alpha * d;

    r = r - alpha * q;

    double delta_old = delta;
    delta = dot(r, r);

    double beta = delta / delta_old;
    d = r + beta * d;
    i++;
}
```

cg.hpp

Optimization #2: Memory Allocation

```
int i = 0;
while (i < imax && delta > ((epsilon * epsilon) * delta0)) {
    VectorType q = A(d);
    double alpha = delta / dot(d, q);
    x = x + alpha * d;

    r = r - alpha * q;

    double delta_old = delta;
    delta = dot(r, r);

    double beta = delta / delta_old;
    d = r + beta * d;
    i++;
}
```

cg.hpp

```
__global__ void add(double * sum, double * a, double * b, int n) {
    int i = threadIdx.x + blockIdx.x * blockDim.x;
    if (i < n) { sum[i] = a[i] + b[i]; }
}

vector operator+(const vector & u, const vector & v) {
    vector out(v.sz);
    int threads_per_block = 256;
    int blocks = (u.sz + threads_per_block - 1) / threads_per_block;
    add<<< blocks, threads_per_block >>>(out.ptr, u.ptr, v.ptr, u.sz);
    return out;
}
```

vector.cu

Optimization #2: Memory Allocation

Problem: lots of short-lived memory allocations in the while loop

Possible remedies:

- Preallocate memory ahead of time, change vector interface to accept argument for temporary storage buffers
 - **Good**: eliminates short-lived allocations, helps perf
 - **Bad: really** hurts code readability, must rewrite algorithms as well
- Lazily-evaluated arithmetic operators (e.g. expression templates)
 - **Good**: eliminates many short-lived allocations, helps perf, readability
 - **Bad**: complicated implementation, compilation time, safety issues
- Custom allocation strategy (e.g. Memory Pools)
 - **Good**: eliminates short-lived allocations, helps perf, readability
 - **Bad**: potentially higher memory high water mark

Optimization #2: Memory Allocation

Problem: lots of short-lived memory allocations in the while loop

Possible remedies:

- Preallocate memory ahead of time, change vector interface to accept argument for temporary storage buffers
 - **Good**: eliminates short-lived allocations, helps perf
 - **Bad**: **really** hurts code readability, must rewrite algorithms as well
- Lazily-evaluated arithmetic operators (e.g. expression templates)
 - **Good**: eliminates many short-lived allocations, helps perf, readability
 - **Bad**: complicated implementation, compilation time, safety issues
- Custom allocation strategy (e.g. Memory Pools)
 - **Good**: eliminates short-lived allocations, helps perf, readability
 - **Bad**: potentially higher memory high water mark

Optimization #2: Memory Allocation

Problem: lots of short-lived memory allocations in the while loop

Possible remedies:

- Preallocate memory ahead of time, change vector interface to accept argument for temporary storage buffers
 - **Good**: eliminates short-lived allocations, helps perf
 - **Bad**: **really** hurts code readability, must rewrite algorithms as well
- Lazily-evaluated arithmetic operators (e.g. expression templates)
 - **Good**: eliminates many short-lived allocations, helps perf, readability
 - **Bad**: complicated implementation, compilation time, safety issues
- Custom allocation strategy (e.g. Memory Pools)
 - **Good**: eliminates short-lived allocations, helps perf, readable, simple
 - **Bad**: potentially higher memory high water mark

Optimization #2: Memory Allocation

Problem: lots of short-lived memory allocations in the while loop

Possible remedies:

- Preallocate memory ahead of time, change vector interface to accept argument for temporary storage buffers
 - **Good:** eliminates short-lived allocations, helps perf
 - **Bad:** really hurts code readability, must rewrite algorithms as well
- Lazily-evaluated arithmetic operators (e.g. expression templates)
 - **Good:** eliminates many short-lived allocations, helps perf, readability
 - **Bad:** complicated implementation, compilation time, safety issues
- **Custom allocation strategy** (e.g. Memory Pools)
 - **Good:** eliminates short-lived allocations, helps perf, readability
 - **Bad:** potentially higher memory high water mark

Let's do the custom allocation strategy, since
the other two are just regular C++ code transformations

Stream-Ordered Allocation

These are **synchronous**, and run on the **default** stream:

```
cudaMalloc(void ** ptr, size_t nbytes)
```

```
cudaFree(void * ptr);
```

These are **asynchronous**, and run on the **specified** stream:

```
cudaMallocAsync(void ** ptr, size_t nbytes, cudaStream_t stream)
```

```
cudaFreeAsync(void * ptr, cudaStream_t stream);
```


Stream-Ordered Allocation

These are **synchronous**, and run on the **default** stream:

```
cudaMalloc(void ** ptr, size_t nbytes)
```

```
cudaFree(void * ptr);
```

These are **asynchronous**, and run on the **specified** stream:

```
cudaMallocAsync(void ** ptr, size_t nbytes, cudaStream_t stream)
```

```
cudaFreeAsync(void * ptr, cudaStream_t stream);
```

(i.e. these behave like kernel launches)

Stream-Ordered Allocation

cudaMallocAsync/cudaFreeAsync can also be combined with
cudaMemPool_t to make memory allocation inside a loop less expensive:

```
cudaMemPool_t mempool;  
cudaDeviceGetDefaultMemPool(&mempool, device);  
uint64_t threshold = UINT64_MAX;  
cudaMemPoolSetAttribute(mempool, cudaMemPoolAttrReleaseThreshold, &threshold);  
for (int i = 0; i < 100; i++) {  
    cudaMallocAsync(&ptr, size, stream);  
    kernel<<< ... , stream>>>(ptr);  
    cudaFreeAsync(ptr, stream);  
    cudaStreamSynchronize(stream);    // Only releases memory down to "threshold" bytes  
}
```

See:

<https://developer.nvidia.com/blog/using-cuda-stream-ordered-memory-allocator-part-1/>

<https://developer.nvidia.com/blog/using-cuda-stream-ordered-memory-allocator-part-2/>

Stream-Ordered Allocation

Allocation Cost, With and Without a Threshold



Figure taken from linked articles

See:

<https://developer.nvidia.com/blog/using-cuda-stream-ordered-memory-allocator-part-1/>

<https://developer.nvidia.com/blog/using-cuda-stream-ordered-memory-allocator-part-2/>

Optimization #2: Memory Allocation

Code Refactoring
(see: [06_profiling/poisson_2](#))

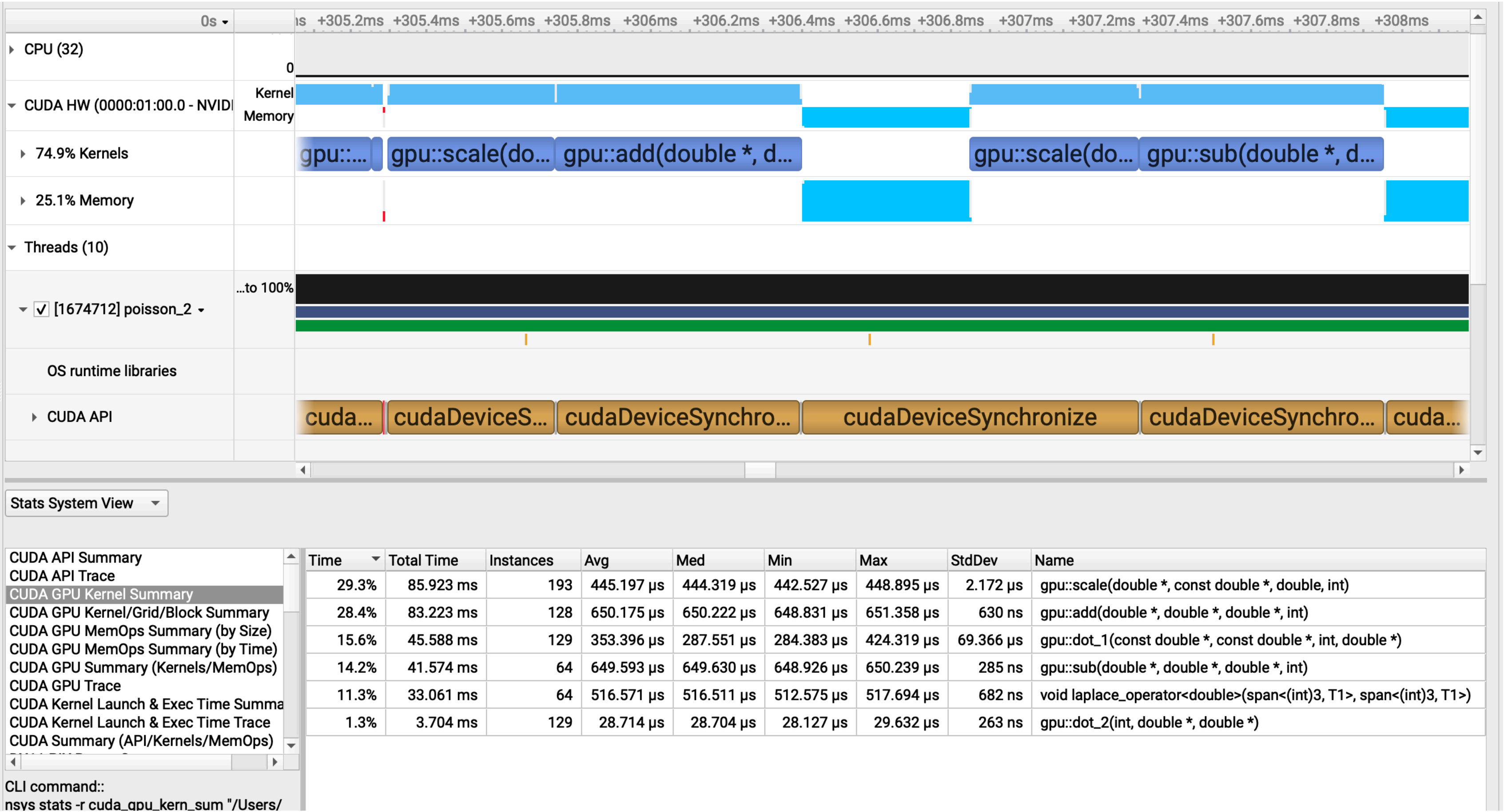
Exercise

- Collect Nsight systems and Nsight Compute profiles from `poisson_2`.
- Inspect the reports with the corresponding GUI applications
 - Which kernels account for the largest fraction of runtime?
 - From the "Summary" tab in Nsight Compute, comment on the compute / memory throughputs of the different kernels.
- (Optional) add NVTX regions to the conjugate gradient algorithm to improve readability of NSight Systems report

```
~/code/Intro-to-CUDA/build$ nsys profile -o poisson_2 -f true ./poisson_2
```

```
~/code/Intro-to-CUDA/build$ ncu -o poisson_2 -f --set full -c 15 --import-source yes ./poisson_2
```

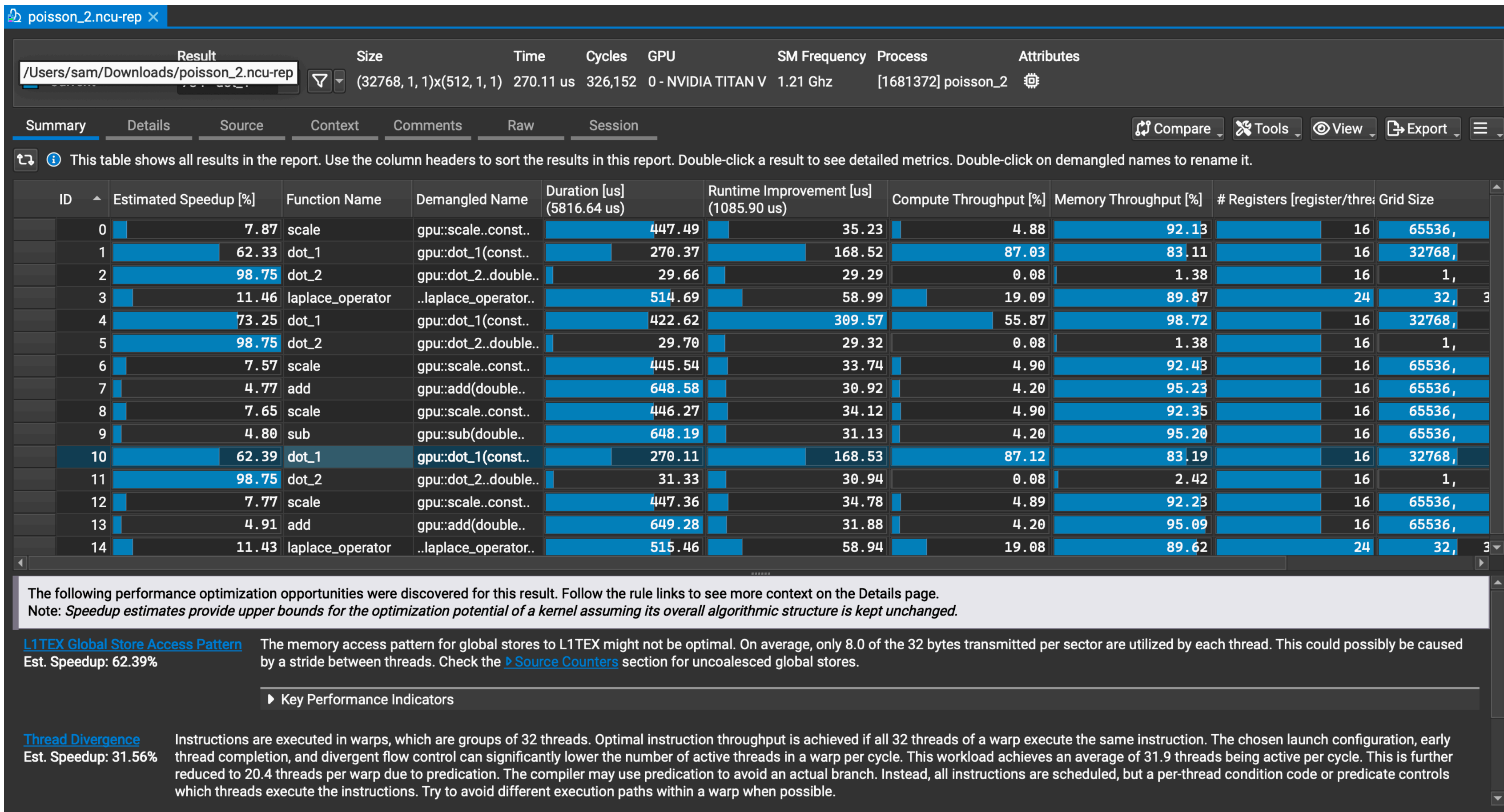
Optimization #2: Memory Allocation



```
~/code/Intro-to-CUDA/build$ ./poisson_2
0 0.166667
1 0.06
2 0.032124
3 0.0195303
4 0.0131258
5 0.00953437
6 0.00713249
7 0.00559492
8 0.00448975
9 0.00367153
10 0.00307865
...
53 0.000138708
54 0.00013376
55 0.000129046
56 0.00012465
57 0.000120391
58 0.000116405
59 0.000112602
60 0.000108948
61 0.00010553
62 0.000102211
63 9.90787e-05
finished in 383.364 ms
```

```
~/code/Intro-to-CUDA/build$ nsys profile -o poisson_2 ./poisson_2
```

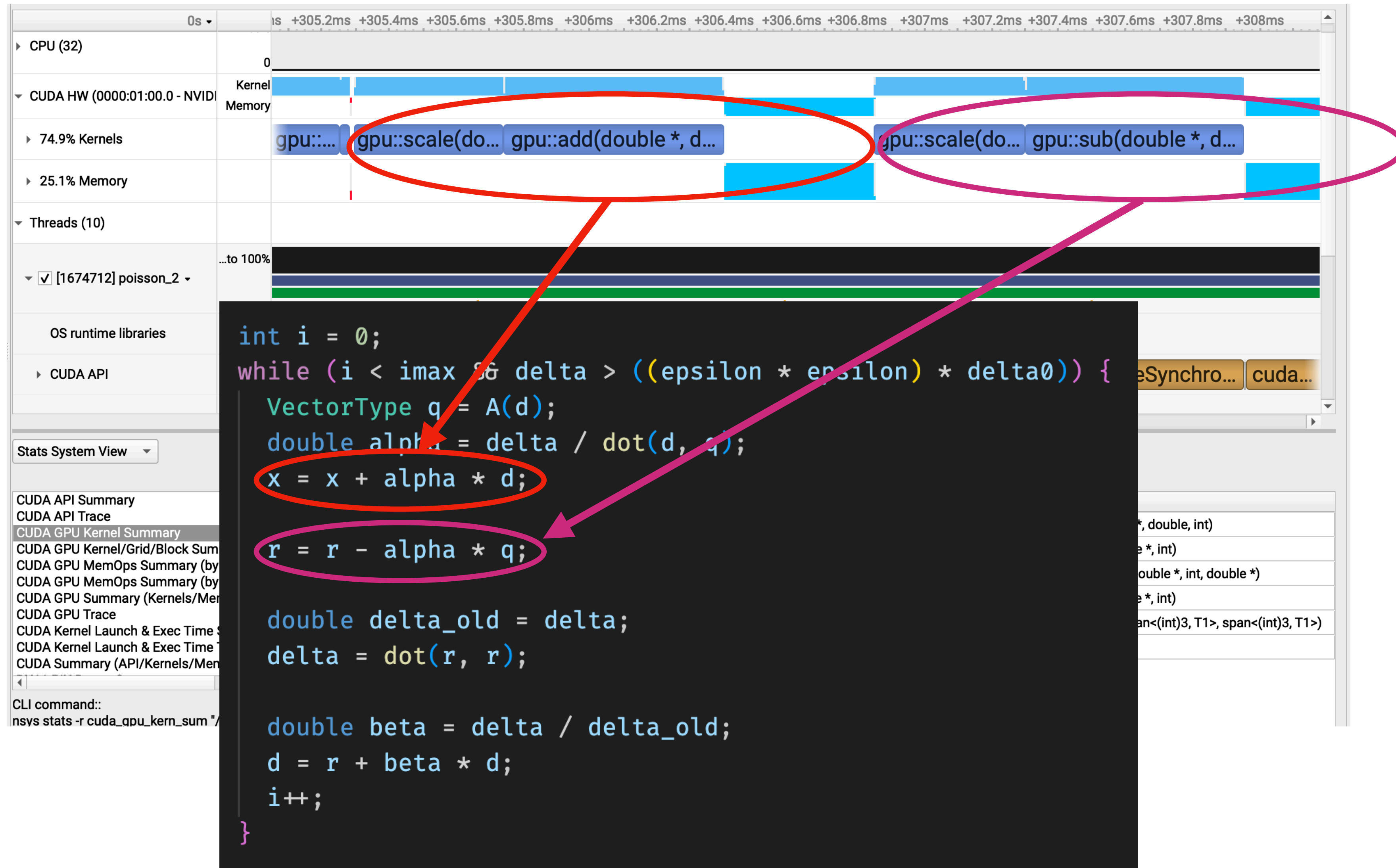

Optimization #2: Memory Allocation



```
~/code/Intro-to-CUDA/build$ ./poisson_2
0 0.166667
1 0.06
2 0.032124
3 0.0195303
4 0.0131258
5 0.00953437
6 0.00713249
7 0.00559492
8 0.00448975
9 0.00367153
10 0.00307865
...
53 0.000138708
54 0.00013376
55 0.000129046
56 0.00012465
57 0.000120391
58 0.000116405
59 0.000112602
60 0.000108948
61 0.00010553
62 0.000102211
63 9.90787e-05
finished in 383.364 ms
```

```
~/code/Intro-to-CUDA/build$ ncu -o poisson_2 -f --set=full -c 15 --import-source yes ./poisson_2
```


Optimization #3: Kernel Fusion



Optimization #3: Kernel Fusion

```
for (int i = 0; i < n; i++) {  
    tmp1[i] = alpha * d[i];  
}  
  
for (int i = 0; i < n; i++) {  
    tmp2[i] = x[i] + tmp1[i];  
}  
  
for (int i = 0; i < n; i++) {  
    x[i] = tmp2[i];  
}
```

4n values read
3n values written
3 kernel launches*

```
for (int i = 0; i < n; i++) {  
    x[i] = x[i] + alpha * d[i];  
}
```

2n values read
1n values written
1 kernel launch*

Aside: Kernel Launch Latency

```
__global__ void empty_kernel() {  
    // no-op  
}  
  
int main() {  
    int n = 1000;  
  
    empty_kernel<<<1,1>>>();  
    cudaDeviceSynchronize();  
  
    timer stopwatch;  
  
    stopwatch.start();  
    for (int i = 0; i < n; i++) {  
        empty_kernel<<<1,1>>>();  
    }  
    cudaDeviceSynchronize();  
    stopwatch.stop();  
  
    std::cout << "launched " << n << " kernels in ";  
    std::cout << stopwatch.elapsed() * 1000.0 << " ms" << std::endl;  
}
```

- Launching a kernel takes a small, but finite, amount of time.
- On my machine this example program reports about 1.5ms to launch 1000 kernels
 - so ~1.5us per launch
 - can be larger for more interesting kernels
 - I usually estimate ~5ms launch latency
- For large kernels, this overhead is "small"
- For small kernels, it matters more
 - CUDA graphs can help in this case

Exercise

See: [06_profiling/poisson_3/readme.txt](#) for more details

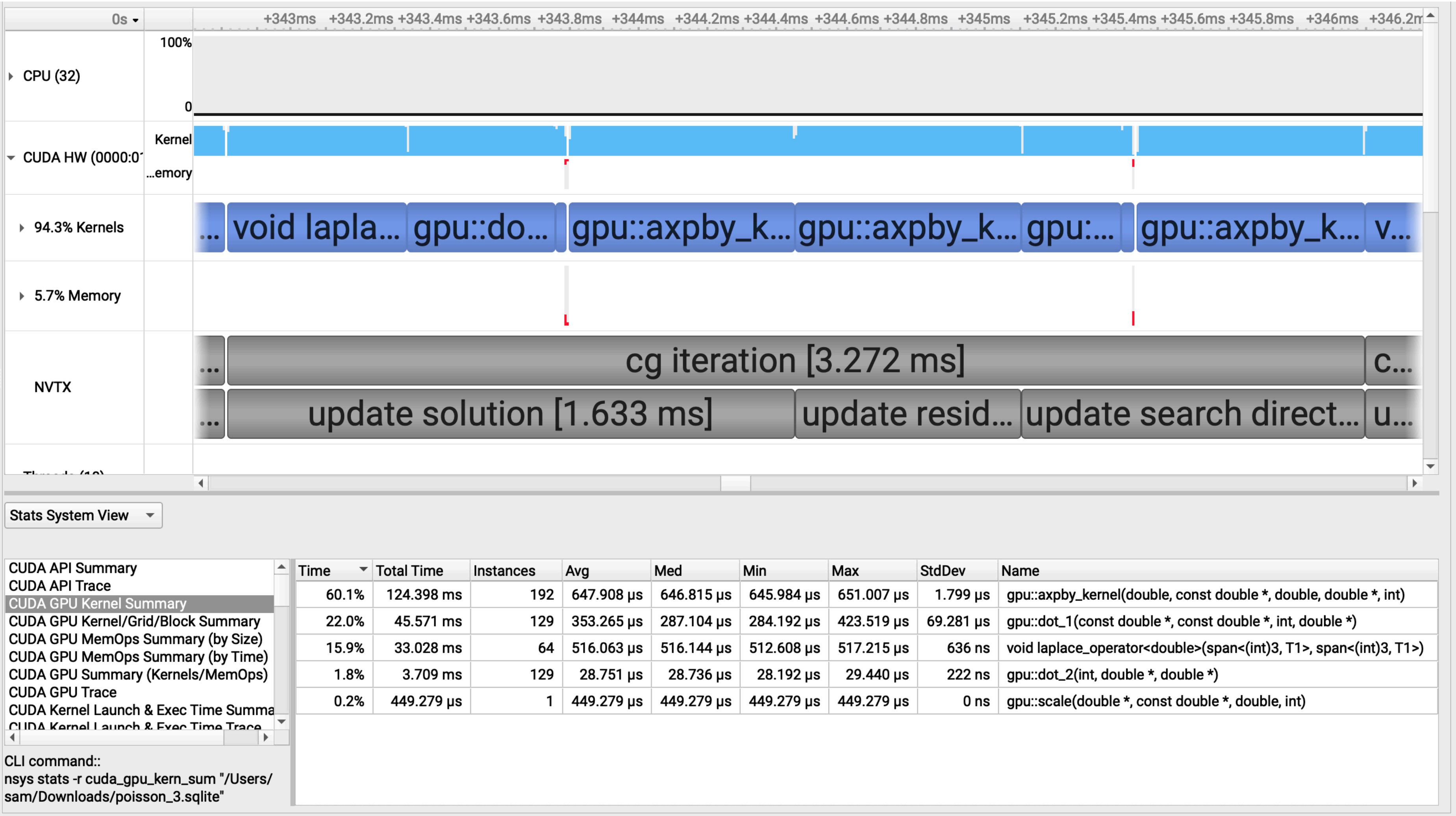
- Implement the `axpby()` function/kernel and use it to replace some of the vector arithmetic in `06_profiling/poisson_3/cg.hpp`
- How does this impact the runtime of the algorithm overall?
- Collect an Nsight systems report of `poisson_3`. Which kernels are the hotspots now?

```
~/code/Intro-to-CUDA/build$ nsys profile -o poisson_3 -f true ./poisson_3
```

```
~/code/Intro-to-CUDA/build$ ncu -o poisson_3 -f --set full -c 15 --import-source yes ./poisson_3
```

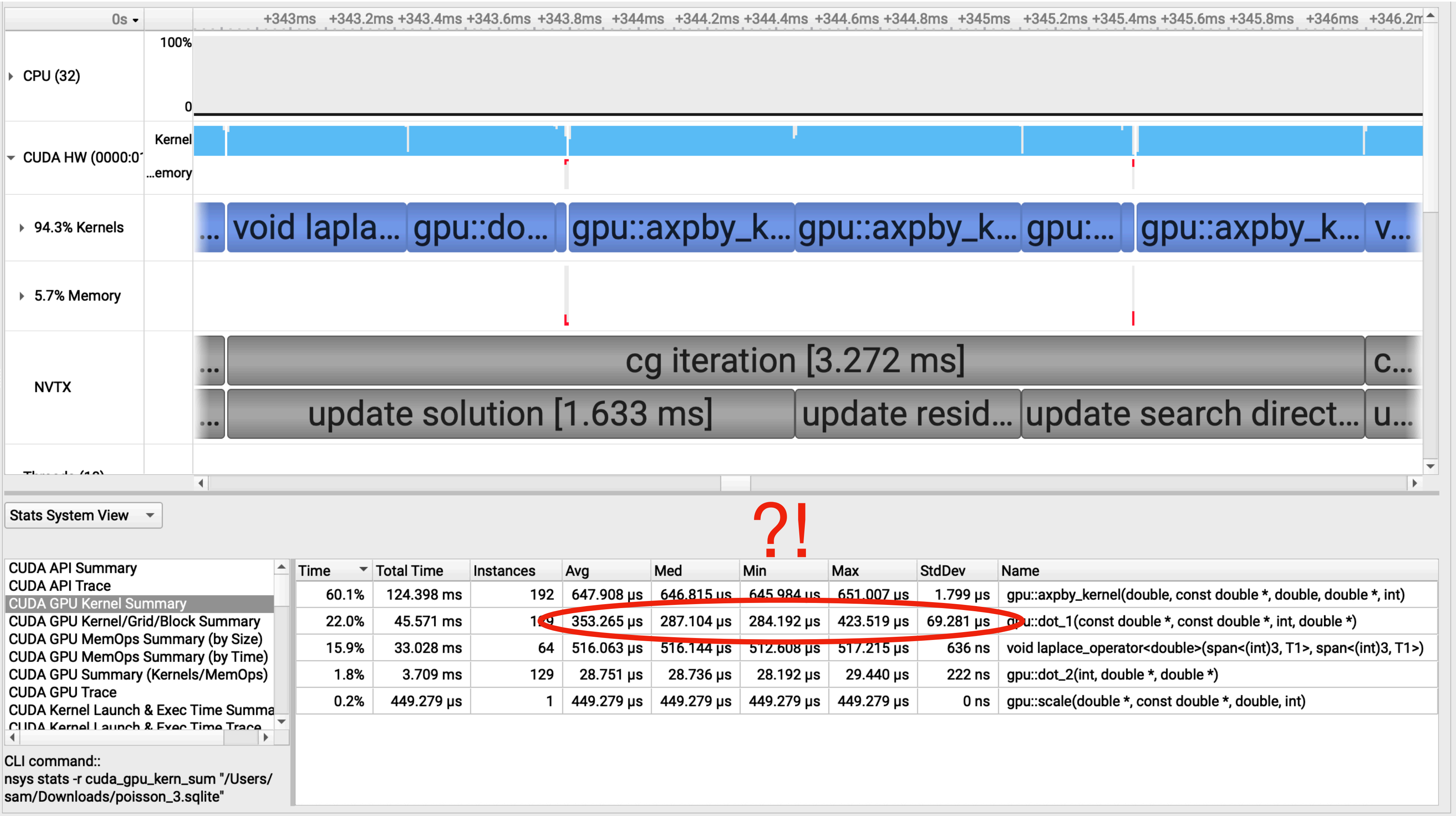

Optimization #3: Kernel Fusion

```
~/code/Intro-to-CUDA/build$ ./poisson_3
0 0.166667
1 0.06
2 0.032124
3 0.0195303
4 0.0131258
5 0.00953437
6 0.00713249
7 0.00559492
8 0.00448975
9 0.00367153
10 0.00307865
...
53 0.000138708
54 0.00013376
55 0.000129046
56 0.00012465
57 0.000120391
58 0.000116405
59 0.000112602
60 0.000108948
61 0.00010553
62 0.000102211
63 9.90787e-05
finished in 210.451 ms
```



Optimization #3: Kernel Fusion

```
~/code/Intro-to-CUDA/build$ ./poisson_3
0 0.166667
1 0.06
2 0.032124
3 0.0195303
4 0.0131258
5 0.00953437
6 0.00713249
7 0.00559492
8 0.00448975
9 0.00367153
10 0.00307865
...
53 0.000138708
54 0.00013376
55 0.000129046
56 0.00012465
57 0.000120391
58 0.000116405
59 0.000112602
60 0.000108948
61 0.00010553
62 0.000102211
63 9.90787e-05
finished in 210.451 ms
```



Optimization #3: Kernel Fusion

Result

Size

Time

Cycles

GPU

SM Frequency

Process

Attributes

Current

707 - axpby_kernel

(65536, 1, 1)x(256, 1, 1)

650.02 us

784,735

0 - NVIDIA TITAN V

1.21 Ghz

[1758999] poisson_3

Summary

Details

Source

Context

Comments

Raw

Session

Compare

Tools

View

Export

This table shows all results in the report. Use the column headers to sort the results in this report. Double-click a result to see detailed metrics. Double-click on demangled names to rename it.

D	Estimated Speedup [%]	Function Name	Demangled Name	Duration [us] (5581.02 us)	Runtime Improvement [us] (1354.97 us)	Compute Throughput [%]	Memory Throughput [%]	# Registers [register/thre	Grid Size
0	7.85	scale	gpu::scale(double...	447.42	35.13	4.88	92.15	16	65536, 1
1	62.35	dot_1	gpu::dot_1(const ...	270.53	168.68	87.06	83.14	16	32768, 1
2	98.75	dot_2	gpu::dot_2(int, do...	29.47	29.10	0.08	1.36	16	1, 1
3	11.46	laplace_operator	void laplace_oper...	514.46	58.98	19.10	89.79	24	32, 32
4	73.23	dot_1	gpu::dot_1(const ...	423.10	309.84	55.89	98.80	16	32768, 1
5	98.75	dot_2	gpu::dot_2(int, do...	29.28	28.91	0.08	1.37	16	1, 1
6	4.96	axpby_kernel	gpu::axpby_kernel...	650.02	32.27	4.20	95.04	16	65536, 1
7	4.96	axpby_kernel	gpu::axpby_kernel...	649.50	32.22	4.19	95.04	16	65536, 1
8	62.39	dot_1	gpu::dot_1(const ...	270.46	168.73	87.12	83.18	16	32768, 1
9	98.75	dot_2	gpu::dot_2(int, do...	29.60	29.23	0.08	1.38	16	1, 1
10	4.99	axpby_kernel	gpu::axpby_kernel...	649.95	32.46	4.20	95.01	16	65536, 1
11	11.46	laplace_operator	void laplace_oper...	515.01	59.00	19.09	89.77	24	32, 32
12	73.23	dot_1	gpu::dot_1(const ...	422.66	309.52	55.90	98.71	16	32768, 1
13	98.75	dot_2	gpu::dot_2(int, do...	29.66	29.29	0.08	1.35	16	1, 1
14	4.86	axpby_kernel	gpu::axpby_kernel...	649.89	31.60	4.20	95.14	16	65536, 1

The following performance optimization opportunities were discovered for this result. Follow the rule links to see more context on the Details page.
Note: Speedup estimates provide upper bounds for the optimization potential of a kernel assuming its overall algorithmic structure is kept unchanged.

Long Scoreboard Stalls

Est. Speedup: 4.96%

On average, each warp of this workload spends 380.6 cycles being stalled waiting for a scoreboard dependency on a L1TEX (local, global, surface, texture) operation. Find the instruction producing the data being waited upon to identify the culprit. To reduce the number of cycles waiting on L1TEX data accesses verify the memory access patterns are optimal for the target architecture, attempt to increase cache hit rates by increasing data locality (coalescing), or by changing the cache configuration. Consider moving frequently used data to shared memory. This stall type represents about 85.8% of the total average of 443.6 cycles between issuing two instructions.

Key Performance Indicators

FP64 Non-Fused Instructions

Est. Speedup: 0.42%

This kernel executes 524288 fused and 524288 non-fused FP64 instructions. By converting pairs of non-fused instructions to their fused, higher-throughput equivalent, the achieved FP64 performance could be increased by up to 25% (relative to its current performance). Check the Source page to identify where this kernel executes FP64 instructions.

What to try next?

```
int i = 0;
while (i < imax && delta > ((epsilon * epsilon) * delta0)) {
    VectorType q = A(d);
    double alpha = delta / dot(d, q);
    axpby(alpha, d, 1.0, x);

    axpby(-alpha, q, 1.0, r);

    double delta_old = delta;
    delta = dot(r, r);

    double beta = delta / delta_old;
    axpby(1.0, r, beta, d);
    i++;
}
```

- Could try more kernel fusion
 - which kernels are still possible to fuse?
 - which remaining kernels would benefit?
- dot product kernels
- Laplace operator kernel

What to try next?

```
int i = 0;
while (i < imax && delta > ((epsilon * epsilon) * delta0)) {
    VectorType q = A(d);
    double alpha = delta / dot(d, q);
    axpby(alpha, d, 1.0, x);

    axpby(-alpha, q, 1.0, r);

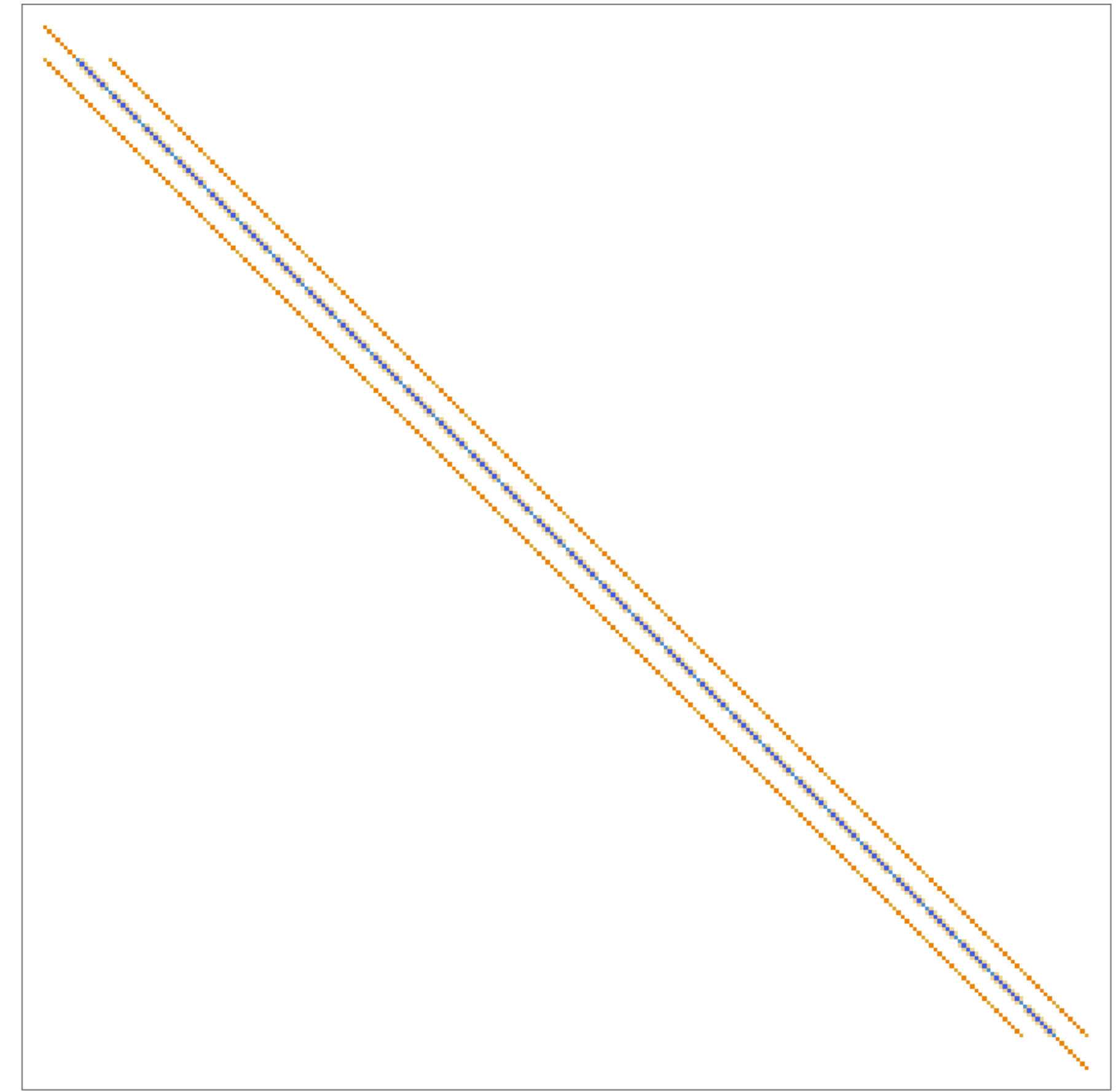
    double delta_old = delta;
    delta = dot(r, r);

    double beta = delta / delta_old;
    axpby(1.0, r, beta, d);
    i++;
}
```

- Could try more kernel fusion
 - which kernels are still possible to fuse?
 - which remaining kernels would benefit?
- dot product kernels
- **Laplace operator kernel**

Optimization #4: SpMV

- 06_profiling/poisson_4 has an implementation that replaces our laplace kernel with cusparseSPMV
 - Also useful as a reference for how to use cuSparse (setting up handle/descriptor objects, etc)
- What is the performance of this version?



matrix representation of laplace operator

Optimization #4: SpMV

```
~/code/Intro-to-CUDA/build$ ./poisson_3
0 0.166667
1 0.06
2 0.032124
3 0.0195303
4 0.0131258
5 0.00953437
6 0.00713249
7 0.00559492
8 0.00448975
9 0.00367153
10 0.00307865
...
53 0.000138708
54 0.00013376
55 0.000129046
56 0.00012465
57 0.000120391
58 0.000116405
59 0.000112602
60 0.000108948
61 0.00010553
62 0.000102211
63 9.90787e-05
finished in 210.451 ms
```

```
~/code/Intro-to-CUDA/build$ ./poisson_4
ndof: 16777216, nnz: 115099600
0 0.166667
1 0.06
2 0.032124
3 0.0195303
4 0.0131258
5 0.00953437
6 0.00713249
7 0.00559492
8 0.00448975
9 0.00367153
10 0.00307865
...
53 0.000138708
54 0.00013376
55 0.000129046
56 0.00012465
57 0.000120391
58 0.000116405
59 0.000112602
60 0.000108948
61 0.00010553
62 0.000102211
63 9.90787e-05
finished in 396.666 ms
```

Optimization #4: SpMV

```
~/code/Intro-to-CUDA/build$ ./poisson_3
0 0.166667
1 0.06
2 0.032124
3 0.0195303
4 0.0131258
5 0.00953437
6 0.00713249
7 0.00559492
8 0.00448975
9 0.00367153
10 0.00307865
...
53 0.000138708
54 0.00013376
55 0.000129046
56 0.00012465
57 0.000120391
58 0.000116405
59 0.000112602
60 0.000108948
61 0.00010553
62 0.000102211
63 9.90787e-05
finished in 210.451 ms
```

Using cuSparseSpMV made our program **slower** by a factor of 2!

What's going on?

```
~/code/Intro-to-CUDA/build$ ./poisson_4
ndof: 16777216, nnz: 115099600
0 0.166667
1 0.06
2 0.032124
3 0.0195303
4 0.0131258
5 0.00953437
6 0.00713249
7 0.00559492
8 0.00448975
9 0.00367153
10 0.00307865
...
53 0.000138708
54 0.00013376
55 0.000129046
56 0.00012465
57 0.000120391
58 0.000116405
59 0.000112602
60 0.000108948
61 0.00010553
62 0.000102211
63 9.90787e-05
finished in 396.666 ms
```


Aside: Arithmetic Intensity

Each kernel:

- Performs some calculations (x FLOP)
- Moves some quantity of data (y GB)
- x / y (units FLOP / Byte) is the kernel's **Arithmetic Intensity**

Each GPU has:

- Compute Units (x' TFLOP/s)
- A Memory Subsystem (y' GB/s)
- x' / y' (units FLOP / Byte) is the GPU's **Arithmetic Intensity**

Aside: Arithmetic Intensity

Comparing these two values tells us whether the kernel is likely to be bottlenecked by Compute vs. Memory

(Kernel A.I.) $\gg$ (GPU A.I.)
Compute Bound

(Kernel A.I.) $\ll$ (GPU A.I.)
Memory Bound

Aside: Arithmetic Intensity

GPU	Compute Throughput in TFLOP/s (FP32 / FP64)	Memory Throughput in GB/s	Arithmetic Intensity in FLOP/B (FP32 / FP64)
GTX Titan V	15 / 7.5	650	23 / 11.5
A30	10 / 5	933	10 / 5
RTX 5090	100 / 1.6	1750	57 / 0.9
B200	80 / 40	8000	10 / 5

Aside: Arithmetic Intensity

Kernel	FLOP	Memory Moved	Arithmetic Intensity in FLOP/B (FP64)
axpby	2*n mult 1*n add	2*n reads 1*n write	0.125
SpMV	nnz mult nnz add (nnz ~ 8n, here)	nnz + n read n write	0.2
laplace operator*	n mult 6*n add	7*n reads 1*n writes	~0.125

Aside: Arithmetic Intensity

So, all these kernels are strongly **memory bound** since their arithmetic intensity is less than 1, but the GPU's tipping point is on the order of 10.

For strongly memory bound kernels, we can estimate the runtime as
$$(\text{\# of bytes moved}) / (\text{\# achieved memory throughput})$$

Aside: Arithmetic Intensity

Kernel	Memory Moved (MB)	Estimated Runtime (us)	Actual Runtime (us)
axpby	~400	666	650
SpMV	~1800	3000	3000
laplace operator*	~1000	1666	500 (?!)
laplace operator	~250	420	500

Times measured on Titan V (Memory Bandwidth ~ 600 GB/s)

Aside: Arithmetic Intensity

"All models are wrong, but some are useful"

Arithmetic intensity is **frequently** useful for quickly estimating bottlenecks and kernel runtimes

It also works for other bandwidth stats (e.g. L1, PCIe, ...)

See also: roofline plots in Nsight Compute

Just for fun: https://cuda.education/fundamentals/arithmetic_intensity/

Optimization #4: SpMV

```
~/code/Intro-to-CUDA/build$ ./poisson_3
```

```
0 0.166667
1 0.06
2 0.032124
3 0.0195303
4 0.0131258
5 0.00963492
6 0.00713249
7 0.00559492
8 0.00448975
9 0.00367153
10 0.00307865
...
```

```
53 0.000138708
54 0.00013376
55 0.000129046
56 0.00012465
57 0.000120391
58 0.000116401
59 0.000112602
60 0.000108948
61 0.00010553
62 0.000102211
63 9.90787e-05
finished in 210.451 ms
```

Using cuSparseSpMV made our program **slower** by a factor of 2!

What's going on?

SpMV fundamentally requires more memory motion
so, it takes longer than our laplace operator kernel

```
~/code/Intro-to-CUDA/build$ ./poisson_4
```

```
ndof: 16777216, nnz: 115099600
0 0.166667
1 0.06
2 0.032124
3 0.0195303
4 0.0131258
5 0.00963492
6 0.00713249
7 0.00559492
8 0.00448975
9 0.00367153
10 0.00307865
...
```

```
53 0.000138708
54 0.00013376
55 0.000129046
56 0.00012465
57 0.000120391
58 0.000116401
59 0.000112602
60 0.000108948
61 0.00010553
62 0.000102211
63 9.90787e-05
finished in 396.666 ms
```

Optimization #5: Blocksize Tuning

See: [06_profiling/poisson_5/laplace_tuning.cu](#)

Try running the laplace operator kernel with the following block sizes:

- 8x8x8
- 16x8x4
- 32x8x2
- 64x8x1
- 128x4x1

How does the performance vary with block size?
What might explain the differences?